

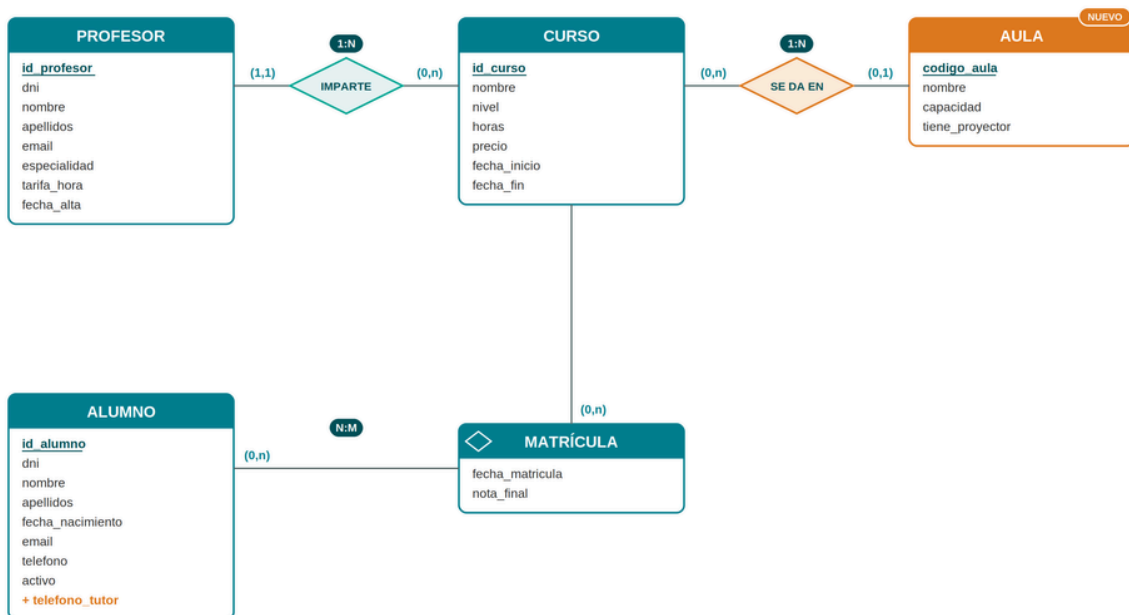
Aulas y cambios de última hora

Academia Código Norte · modificar tablas que ya existen

Duración	Qué vas a aprender	Necesitas
1 sesión (~1 h)	ALTER TABLE: añadir columnas y claves ajenas a tablas con datos. ON DELETE SET NULL	Haber terminado la práctica 3

Dónde estamos

La directora quiere saber en qué **aula** se da cada curso. Además, para los alumnos menores de edad, necesita el **teléfono de un tutor**. Pero **curso** y **alumno** ya existen y tienen datos: no vamos a borrarlas, las vamos a **modificar**.



En naranja, lo nuevo de esta práctica.

Lo nuevo de hoy

- **ALTER TABLE:** cambia una tabla que ya existe sin perder sus datos. En la vida real es lo más habitual: las tablas casi nunca se crean de nuevo.
- **ON DELETE SET NULL:** si se borra un aula, sus cursos **no** se borran; se quedan sin aula (la columna se vacía).

La tabla AULA (nueva)

```
CREATE TABLE aula (
    codigo_aula    VARCHAR(4)    PRIMARY KEY,
    nombre         VARCHAR(40)   NOT NULL,
    capacidad      INT           NOT NULL DEFAULT 20 CHECK (capacidad BETWEEN 1 AND 40),
    tiene_proyector BOOLEAN      NOT NULL DEFAULT FALSE
);
```

Qué guarda cada columna:

Columna	Tipo	Reglas	Qué guarda
codigo_aula	VARCHAR(4)	PRIMARY KEY	Código del aula, por ejemplo A101
nombre	VARCHAR(40)	NOT NULL	Nombre del aula
capacidad	INT	NOT NULL DEFAULT 20 CHECK (capacidad BETWEEN 1 AND 40)	Plazas (20 si no se dice nada)
tiene_proyector	BOOLEAN	NOT NULL DEFAULT FALSE	Si tiene proyector

Pista

La clave primaria **no** es un número: es el código que ya usa la academia. Por eso es un texto, `VARCHAR(4)`, y no un número.

Los cambios en tablas que ya existen

Estos son los tres cambios, **en este orden**. Código completo:

```
ALTER TABLE curso ADD COLUMN codigo_aula VARCHAR(4);
```

```
ALTER TABLE curso ADD CONSTRAINT fk_curso_aula
```

```
    FOREIGN KEY (codigo_aula) REFERENCES aula (codigo_aula) ON DELETE SET NULL;
```

```
ALTER TABLE alumno ADD COLUMN telefono_tutor VARCHAR(15);
```

Qué hace cada uno:

#	Tabla	Cambio	Qué hace
1	curso	ADD COLUMN	Añade la columna codigo_aula (VARCHAR(4), sin reglas: puede quedar vacía). Los cursos que ya existen la tendrán vacía.
2	curso	ADD CONSTRAINT	Convierte codigo_aula en clave ajena hacia aula. Si se borra un aula, el curso se queda sin aula (SET NULL).
3	alumno	ADD COLUMN	Añade la columna telefono_tutor (VARCHAR(15), sin reglas).

¿Por qué ese orden?

No puedes crear la clave ajena (2) hasta que exista la columna (1). Y la tabla `aula` tiene que existir antes que las dos.

Paso a paso

Hoy tendrás **7 consultas guardadas** en el SQL Editor, cada una con su nombre (igual que en la práctica 1):

Nombre de la consulta	Qué tiene dentro
-----------------------	------------------

P4 · 1 · Tabla aula	El CREATE TABLE aula
P4 · 2 · Columna codigo_aula	ALTER TABLE curso ADD COLUMN codigo_aula ...
P4 · 3 · Clave ajena aula	ALTER TABLE curso ADD CONSTRAINT fk_curso_aula ...
P4 · 4 · Columna telefono_tutor	ALTER TABLE alumno ADD COLUMN telefono_tutor ...
P4 · 5 · Datos de prueba	Los INSERT de aulas y los UPDATE
P4 · 6 · Intenta romperla	Las dos pruebas que deben fallar
P4 · 7 · Prueba SET NULL	El bloque BEGIN ... ROLLBACK

1. Crea cada consulta **en el orden de la tabla**: **New query**, pega, **Run** y, si sale «Success», renómbrala (tres puntos → **Rename query**).
2. En el **Table Editor**, abre **curso**: tiene una columna nueva, **codigo_aula**, **vacía** en todos los cursos. Los datos de antes siguen ahí.
3. Ejecuta las pruebas de **Intenta romperla** una a una (selecciona cada una y pulsa **Run**) y la de **SET NULL**.

Datos de prueba

```
INSERT INTO aula (codigo_aula, nombre, capacidad, tiene_proyector) VALUES
```

```
('A101', 'Aula Turing', 24, TRUE),
```

```
('A102', 'Aula Lovelace', 16, TRUE),
```

```
('B201', 'Aula Hopper', DEFAULT, FALSE);
```

```
UPDATE curso SET codigo_aula = 'A101' WHERE id_curso = 1;
```

```
UPDATE curso SET codigo_aula = 'A102' WHERE id_curso = 2;
```

```
UPDATE curso SET codigo_aula = 'B201' WHERE id_curso = 3;
```

```
UPDATE alumno SET telefono_tutor = '600999888' WHERE id_alumno = 1;
```

Comprueba

aula tiene 3 filas. El aula Hopper tiene 20 plazas aunque no lo escribimos (DEFAULT).

En `curso`, los cursos 1, 2 y 3 tienen aula. El 4 (Next.js) todavía no.

Intenta romperla

#	Sentencia	Debe fallar porque...
1	<pre>INSERT INTO aula (codigo_aula, nombre, capacidad) VALUES ('C301', 'Aula Grande', 60);</pre>	Ninguna aula pasa de 40 plazas (CHECK)
2	<pre>UPDATE curso SET codigo_aula = 'Z999' WHERE id_curso = 4;</pre>	No existe el aula Z999 (fk_curso_aula)

Comprueba el SET NULL

Ejecuta este bloque **entero**. `BEGIN` y `ROLLBACK` hacen que el borrado se deshaga al final, así no pierdes el aula:

```
BEGIN;
```

```
DELETE FROM aula WHERE codigo_aula = 'B201';
```

```
SELECT id_curso, nombre, codigo_aula FROM curso WHERE id_curso = 3;
```

```
ROLLBACK;
```

Comprueba

El curso 3 (HTML y CSS) sigue existiendo, pero su `codigo_aula` sale vacío.

Piensa

¿Qué habría pasado si hubiéramos borrado la tabla `curso` y la hubiéramos creado otra vez con la columna nueva?

¿Por qué para el aula usamos `SET NULL` y para el profesor usamos `RESTRICT`?

¿Te has equivocado? Empieza de nuevo

Ejecuta esto y vuelve a lanzar tu código de esta práctica desde el principio. Solo borra lo de **esta** práctica: lo de las anteriores no se toca.

```
DROP TABLE IF EXISTS aula CASCADE;  
  
ALTER TABLE curso DROP COLUMN IF EXISTS codigo_aula;  
  
ALTER TABLE alumno DROP COLUMN IF EXISTS telefono_tutor;
```

Qué entregas

Captura de la lista de consultas con las de la práctica 4 ([P4](#) · ...: la tabla aula y los tres ALTER TABLE, cada uno en su consulta) y captura del resultado del SET NULL.

Anclaje curricular · RD 405/2023 · Módulo 0484 · RA2 · **criterios b, c, d, e y h**